

DAY-2 ASSIGNMENT [DSA]

UDAY CHIRRA

2303A510G2

1. Remove Element

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int k = 0;
        for (int num : nums) {
            if (num != val) {
                nums[k++] = num;
            }
        }
        return k;
    }
}
```

2. Reverse String

```
class Solution {
    public void reverseString(char[] s) {
        int left = 0, right = s.length - 1;

        while (left < right) {
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;
            left++;
            right--;
        }
    }
}
```

3. Merge Sorted Array

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;
```

```

while (j >= 0) {
    if (i >= 0 && nums1[i] > nums2[j]) {
        nums1[k--] = nums1[i--];
    } else {
        nums1[k--] = nums2[j--];
    }
}
}
}

```

4. Sort Colors

```

class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low, mid);
                low++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                swap(nums, mid, high);
                high--;
            }
        }
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}

```

5. Next Permutation

```

class Solution {
    public void nextPermutation(int[] nums) {
        int i = nums.length - 2;

        while (i >= 0 && nums[i] >= nums[i + 1]) {
            i--;
        }

        if (i >= 0) {
            int j = nums.length - 1;

            while (nums[j] <= nums[i]) {
                j--;
            }

            swap(nums, i, j);
        }

        reverse(nums, i + 1, nums.length - 1);
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    private void reverse(int[] nums, int left, int right) {
        while (left < right) {
            swap(nums, left, right);
            left++;
            right--;
        }
    }
}

```

6. Find the Duplicate Number

```

class Solution {
    public int findDuplicate(int[] nums) {

```

```

int slow = nums[0];
int fast = nums[0];

do {
    slow = nums[slow];
    fast = nums[nums[fast]];
} while (slow != fast);

slow = nums[0];

while (slow != fast) {
    slow = nums[slow];
    fast = nums[fast];
}

return slow;
}
}

```

7. First Missing Positive

```

class Solution {
public int firstMissingPositive(int[] nums) {
    int n = nums.length;

    for (int i = 0; i < n; i++) {
        while (nums[i] > 0 &&
            nums[i] <= n &&
            nums[nums[i] - 1] != nums[i]) {

            int temp = nums[i];
            nums[i] = nums[temp - 1];
            nums[temp - 1] = temp;
        }
    }

    for (int i = 0; i < n; i++) {
        if (nums[i] != i + 1) {
            return i + 1;
        }
    }
}
}

```

```

        return n + 1;
    }
}

```

8. Trapping Rain Water

```

class Solution {
    public int trap(int[] height) {
        int left = 0;
        int right = height.length - 1;
        int leftMax = 0;
        int rightMax = 0;
        int water = 0;

        while (left < right) {
            if (height[left] <= height[right]) {
                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    water += leftMax - height[left];
                }
                left++;
            } else {
                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    water += rightMax - height[right];
                }
                right--;
            }
        }

        return water;
    }
}

```

9. Rotate Array

```

class Solution {
    public void rotate(int[] nums, int k) {

```

```
int n = nums.length;
k = k % n;

reverse(nums, 0, n - 1);
reverse(nums, 0, k - 1);
reverse(nums, k, n - 1);
}

private void reverse(int[] nums, int left, int right) {
    while (left < right) {
        int temp = nums[left];
        nums[left] = nums[right];
        nums[right] = temp;
        left++;
        right--;
    }
}
}
```